

Fichas de Atención

De las RNN a la atención, la arquitectura Transformer, su costo, su implementación en PyTorch y cómo leer la atención de BERT y GPT-2. 56 fichas en 8 temas.

Brandon Werner Rivera Cabrera · 23088 · Universidad del Valle de Guatemala



atención causal: cada token solo ve hacia atrás

● De las RNN a la atención

7 fichas

¿Qué es el cuello de botella del seq2seq con RNN?

El encoder comprime toda la oración de entrada en un solo vector de tamaño fijo, su último estado oculto, y el decoder solo ve ese vector. Con oraciones largas la información del inicio se diluye en ese vector y la calidad de la traducción cae.

¿Cómo resuelve la atención de Bahdanau el cuello de botella?

El decoder ya no depende de un solo vector: en cada paso calcula un puntaje entre su estado y cada estado del encoder, los normaliza con softmax y usa su promedio ponderado como contexto. Cada palabra de salida puede mirar directamente la parte relevante de la entrada.

Atención aditiva frente a multiplicativa

Aditiva (Bahdanau): $\text{puntaje} = v^T \tanh(W_1 h_i + W_2 s_t)$, una pequeña red de una capa.

Multiplicativa (Luong): $\text{puntaje} = s_t^T h_i$ o $s_t^T W h_i$. La multiplicativa es más eficiente porque todos los puntajes salen de una multiplicación de matrices, que las GPU ejecutan con kernels muy optimizados.

La analogía de query, key y value

Es una búsqueda suave en un diccionario. La query es lo que se busca, cada key es la etiqueta de una entrada y cada value su contenido. En vez de devolver solo la entrada cuya llave coincide, se devuelve un promedio de todos los valores ponderado por qué tanto se parece cada key a la query.

● Scaled dot-product attention

7 fichas

Fórmula de scaled dot-product attention

$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$. Cada fila de la softmax es la distribución de atención de un token sobre todos los demás y suma 1.

Si q y k tienen componentes independientes de media 0 y varianza 1, ¿cuál es la varianza de q·k?

d_k . $q \cdot k = \sum q_i k_i$ y cada producto tiene media 0 y varianza 1, así que la suma de d_k términos independientes tiene varianza d_k . Al dividir entre $\sqrt{d_k}$ la varianza vuelve a 1.

¿Qué le pasa a la softmax sin el escalamiento?

Con d_k grande los puntajes tienen desviación $\sqrt{d_k}$, la softmax se satura y casi todo el peso cae en una sola posición. En ese régimen su jacobiano $\text{diag}(p) - pp^T$ es casi cero y los gradientes se desvanecen.

● Multi-head attention

6 fichas

¿Qué se gana con h cabezas de dimensión d_{model}/h ?

Cada cabeza aprende su propia proyección y puede atender a un tipo de relación distinto: sintaxis, posición, coreferencia. Una sola cabeza promedia todo en una sola distribución y pierde esa diversidad.

¿Cambia el número de parámetros al usar más cabezas?

No. W_Q, W_K, W_V y W_O siguen siendo $d_{\text{model}} \times d_{\text{model}}$: con sesgos son $4d^2 + 4d$ parámetros sin importar h. Con $d = 512$ son 1 050 624, igual con 1, 8 o 16 cabezas.

Configuración del Transformer base de Vaswani et al.

$d_{\text{model}} = 512$, $h = 8$ cabezas con $d_k = d_v = 64$, $d_{\text{ff}} = 2048$, 6 capas en encoder y en decoder, dropout 0.1 y unos 65 M de parámetros.

¿Para qué sirve W_O ?

Las salidas de las h cabezas se concatenan y W_O las mezcla de vuelta a d_{model} . Es lo que permite combinar la información que cada cabeza extrajo por separado.

● Codificación posicional

5 fichas

¿Por qué la self-attention no conoce el orden de los tokens?

Es equivariante a permutaciones: si se permutan los tokens de entrada, la salida es la misma permutación de la salida original, porque la atención solo usa productos punto entre pares sin saber dónde está cada uno. Sin codificación posicional, «perro muerde a hombre» y «hombre muerde a perro» darían las mismas representaciones.

Codificación posicional sinusoidal

$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{2i/d})$ y $\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{2i/d})$. Cada par de dimensiones es un reloj con una frecuencia distinta. Se suma al embedding del token.

¿De dónde salen Q, K y V en self-attention y en cross-attention?

En self-attention las tres son proyecciones lineales de la misma secuencia: $Q = XW_Q$, $K = XW_K$, $V = XW_V$. En cross-attention del decoder, Q sale del decoder y K y V salen de la salida del encoder.

¿Qué limitaciones de las RNN elimina el Transformer?

El procesamiento secuencial, porque todas las posiciones se calculan en paralelo; las dependencias largas, porque cualquier par de tokens está a un paso de distancia y no a n pasos; y el desvanecimiento del gradiente a lo largo del tiempo, porque el gradiente no tiene que atravesar n multiplicaciones por la misma matriz.

Tabla 1 de Vaswani et al.: self-attention frente a RNN

CAPA	COSTO POR CAPA	OPS SECUENCIALES	CAMINO MÁXIMO
Self-attention	$O(n^2 d)$	$O(1)$	$O(1)$
Recurrente	$O(n d^2)$	$O(n)$	$O(n)$
Convolutacional	$O(k n d^2)$	$O(1)$	$O(\log_k n)$

Self-attention es más barata que una RNN mientras $n < d$, que es lo común en oraciones.

¿Cómo funciona una máscara de atención?

Se suma $-\infty$ a los puntajes de las posiciones prohibidas antes de la softmax, así su peso queda exactamente en 0. La máscara causal prohíbe mirar a la derecha; la de padding prohíbe atender a tokens de relleno.

Ejercicio a mano: $Q = K = [[1,0],[0,1],[1,1]]$, $d_k = 2$. ¿Puntajes escalados?

$QK^T = [[1,0,1],[0,1,1],[1,1,2]]$. Dividido entre $\sqrt{2}$: $[[0.707, 0, 0.707], [0, 0.707, 0.707], [0.707, 0.707, 1.414]]$.

Ejercicio a mano: pesos y salida con $V = [[1,2],[3,4],[5,6]]$

Softmax por fila: $[0.401, 0.198, 0.401]$, $[0.198, 0.401, 0.401]$ y $[0.248, 0.248, 0.504]$. Salida = pesos · V: $[3, 4]$, $[3.407, 4.407]$ y $[3.511, 4.511]$. Con máscara causal la primera fila es $[1, 0, 0]$ y su salida es $[1, 2]$.

¿Por qué la atención de un token consigo mismo suele ser alta en self-attention?

Porque q y k salen del mismo vector x; si W_Q y W_K son parecidas, $q \cdot k$ de un token consigo mismo tiende a ser grande. El modelo aprende a evitarlo cuando no le sirve, por eso hay cabezas que miran al token anterior o al siguiente.

Tamaños de BERT-base y GPT-2 small

	BERT-BASE	GPT-2 SMALL
Capas	12	12
Cabezas	12	12
d_{model}	768	768
Parámetros	110 M	124 M
Vocabulario	30 522 WordPiece	50 257 BPE
Contexto	512	1 024

Encoder, decoder y encoder-decoder

BERT es solo encoder: atención bidireccional, entrenado con masked LM. GPT-2 es solo decoder: atención causal, entrenado prediciendo el siguiente token. El Transformer original es encoder-decoder, con cross-attention del decoder hacia el encoder.

¿Qué propiedad tiene la codificación sinusoidal?

$\text{PE}(\text{pos} + k)$ es una rotación fija de $\text{PE}(\text{pos})$ que solo depende de k, así que el producto punto $\text{PE}(p) \cdot \text{PE}(p + k)$ es igual para cualquier p. Eso facilita que el modelo aprenda posiciones relativas y extrapole a longitudes no vistas.

Embeddings posicionales aprendidos

Una tabla con un vector entrenable por posición, como en BERT y GPT-2. Rinden parecido a los sinusoidales, pero no generalizan más allá de la longitud máxima vista en entrenamiento: 512 en BERT y 1 024 en GPT-2.

● Bloque Transformer

6 fichas

Partes de un bloque encoder

Self-attention multi-cabeza, luego una red feed-forward por posición (dos lineales con una activación en medio, de d a 4d y de vuelta a d). Cada sub-bloque tiene una conexión residual y una LayerNorm.

¿Para qué sirven las conexiones residuales?

Dejan pasar la señal y el gradiente sin transformación: $x + F(x)$. Permiten apilar muchas capas sin que el gradiente se desvanezca y hacen que cada capa aprenda una corrección sobre su entrada.

¿Qué hace la red feed-forward por posición?

Aplica el mismo MLP a cada token por separado. La atención mezcla información entre posiciones y la FFN la procesa dentro de cada posición; contiene alrededor de dos tercios de los parámetros de cada bloque.

● Complejidad y atención eficiente

7 fichas

Costo de la self-attention con la longitud n

Tiempo $O(n^2d)$: QK^T y la multiplicación por V hacen $2n^2d$ MACs cada una. Memoria $O(n^2)$ por cabeza para la matriz de pesos. Duplicar la secuencia cuadruplica el tiempo y la memoria.

¿Qué hace FlashAttention y qué sacrifica?

Calcula la atención exacta por bloques dentro de la memoria rápida de la GPU con una softmax en línea, sin escribir la matriz $n \times n$ en memoria global. La memoria baja a $O(n)$ y es varias veces más rápida, pero los FLOPs siguen siendo $O(n^2)$ y necesita kernels específicos del hardware.

Atención por ventanas o dispersa

Cada token atiende solo a una ventana local de w vecinos, más algunos tokens globales en Longformer o BigBird. El costo baja a $O(n \cdot w)$, a cambio de que dos tokens lejanos solo se comuniquen a través de varias capas o de los tokens globales.

Atención lineal

Reemplaza softmax(QK^T) por $\phi(Q)\phi(K)^T$ y reordena: $\phi(Q)(\phi(K)^TV)$ cuesta $O(n \cdot d^2)$. Sacrifica exactitud: el kernel aproxima mal la softmax y la atención queda menos concentrada, lo que suele bajar la calidad.

● PyTorch y Hugging Face

8 fichas

`F.scaled_dot_product_attention` y sus parámetros

Recibe query, key y value de forma (..., n, d) y elige internamente el kernel más rápido, como FlashAttention o memory-efficient. `attn_mask`: booleana (True = participa) o flotante, que se suma a los puntajes. `dropout_p`: dropout sobre los pesos. `is_causal`: aplica máscara causal sin construirla. `scale`: reemplaza $1/\sqrt{d}$.

`nn.MultiheadAttention`: parámetros del constructor

`embed_dim` y `num_heads`, con `embed_dim` divisible entre `num_heads`; `batch_first` para usar (batch, n, d), que por defecto es False, o sea (n, batch, d); `kdimm` y `vdimm` cuando key y value tienen otra dimensión, como en cross-attention.

Argumentos del forward de `nn.MultiheadAttention`

`key_padding_mask`: (batch, m), True marca padding a ignorar. `attn_mask`: (n, m) o (batch-h, n, m). `need_weights`: devolver o no los pesos. `average_attn_weights`: True promedia las cabezas; False devuelve (batch, h, n, m).

Trampa: el significado de True en las máscaras booleanas

En `nn.MultiheadAttention`, True en `attn_mask` y `key_padding_mask` significa **prohibido**. En `F.scaled_dot_product_attention`, True en `attn_mask` significa **permitido**. Confundirlas invierte la máscara.

● Interpretación de la atención

10 fichas

Patrones típicos de las cabezas de BERT

Clark et al. (2019): cabezas que miran al token anterior o siguiente, cabezas que atienden casi todo a [SEP] o a la puntuación, cabezas con atención amplia en las primeras capas y algunas que siguen relaciones sintácticas como objeto directo o determinante, y coreferencia.

¿Por qué tantas cabezas de BERT atienden a [SEP]?

Clark et al. lo interpretan como una operación nula: cuando una cabeza no encuentra nada relevante, manda su atención a un token que está en todas las entradas y cuyo value aporta poco. Al quitar esa atención el gradiente cambia poco, señal de que no es información útil.

Post-LN frente a Pre-LN

Post-LN, del artículo original: $x \leftarrow \text{LN}(x + F(x))$. **Pre-LN**, usado hoy: $x \leftarrow x + F(\text{LN}(x))$ más una LN final. En Post-LN los gradientes cerca de la salida son grandes al inicio y hace falta warmup; Pre-LN deja un camino residual limpio y entrena estable sin warmup (Xiong et al., 2020).

¿Qué hace LayerNorm?

Normaliza cada token sobre sus d dimensiones, a media 0 y varianza 1, y aplica una escala γ y un desplazamiento β aprendidos. A diferencia de BatchNorm no depende del lote, por eso funciona con secuencias de largo variable.

¿Por qué el decoder usa una máscara causal?

Al entrenar se predicen todas las posiciones en paralelo; si el token i pudiera ver los tokens $j > i$ estaría viendo la respuesta. La máscara triangular deja que cada posición atienda solo a las anteriores, como en la generación real.

Documentos de 100 000 tokens: problema y alternativas

La matriz de atención tendría 10^{10} entradas por cabeza, unos 40 GB en fp32, y el tiempo crece con n^2 . Alternativas: FlashAttention con paralelismo de secuencia, atención por ventanas con tokens globales, atención lineal o de estado como Mamba, recuperación de fragmentos relevantes (RAG) o procesar por bloques con memoria recurrente.

¿Por qué la atención permitió entrenar modelos mucho más grandes que las RNN?

Todas las posiciones se procesan en paralelo con multiplicaciones de matrices, que es lo que mejor hacen las GPU, y el camino entre dos tokens es $O(1)$, así que las dependencias largas se aprenden sin pasar el gradiente por n pasos. El costo cuadrático es aceptable mientras n no sea enorme.

¿Cuándo preferir una RNN o una CNN sobre un Transformer?

Con secuencias muy largas o streaming en dispositivos limitados, donde el costo $O(n)$ por token y la memoria constante de una RNN importan; con pocos datos, porque el sesgo inductivo de localidad de una CNN o de recurrencia de una RNN generaliza mejor; y en señales locales como audio crudo o series de tiempo cortas.

`nn.TransformerEncoderLayer`

Un bloque encoder completo. `d_model`, `nhead`, `dim_feedforward` (2048 por defecto), `dropout` (0.1), `activation`, `batch_first` y `norm_first`: False es Post-LN y True es Pre-LN. Se apila con `nn.TransformerEncoder`.

¿Qué devuelve `generate_square_subsequent_mask(n)`?

Una máscara flotante $n \times n$ con 0 en y debajo de la diagonal y $-\infty$ arriba, lista para sumarse a los puntajes. Para $n = 3$: $\begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}$.

```
mask = nn.Transformer.generate_square_subsequent_mask(n)
```

¿Cómo obtener las atenciones de un modelo de Hugging Face?

Con `output_attentions=True` la salida trae `attentions`, una tupla con un tensor por capa de forma (batch, cabezas, n, n). Hace falta `attn_implementation="eager"`, porque los kernels sdpa y flash no materializan los pesos.

```
bert = AutoModel.from_pretrained("bert-base-uncased",
    output_attentions=True, attn_implementation="eager")
A = torch.stack(bert(**enc).attentions) # (capas, batch, h, n, n)
```

¿Cómo se apaga una cabeza de GPT-2 en transformers 5?

Ya no existe `head_mask`. Se registra un forward pre-hook en `attn.c_proj` de la capa y se ponen en cero las columnas de esa cabeza en la entrada, que son las posiciones $h \cdot d_h$ a $(h+1) \cdot d_h$ de la concatenación de cabezas.

¿Qué son los attention sinks?

Xiao et al. (2023): en modelos causales como GPT-2 o LLaMA, el primer token recibe una fracción enorme de la atención en las capas profundas aunque no sea importante. La softmax obliga a repartir peso que sume 1 y el primer token, visible para todos, funciona como depósito. StreamingLLM conserva esos primeros tokens junto a una ventana reciente para generar textos muy largos.

¿Cómo se verifica que la atención de GPT-2 es causal?

Todos los pesos por encima de la diagonal deben ser exactamente 0 en todas las capas y cabezas, y cada fila debe sumar 1. La primera fila es siempre $[1, 0, \dots, 0]$ porque el primer token solo puede verse a sí mismo.

Entropía de la atención

$H = -\sum p \log p$ de cada fila; es 0 si toda la atención va a un token y $\log n$ si es uniforme. Se divide entre $\log$ del número de posiciones visibles para comparar filas: n en BERT y $i + 1$ en la fila i de GPT-2. Una entropía baja indica atención concentrada.

¿Todas las cabezas son útiles?

No necesariamente. Michel et al. (2019) y Voita et al. (2019) mostraron que se puede quitar una gran parte de las cabezas con poca pérdida de calidad; unas pocas, como las posicionales o sintácticas, concentran la importancia. Se mide apagando una cabeza a la vez y viendo cuánto sube la pérdida.

Jain y Wallace (2019): «Attention is not Explanation»

Encontraron que los pesos de atención se correlacionan poco con medidas de importancia por gradiente o por eliminación de tokens, y que se pueden construir distribuciones de atención muy distintas que producen la misma predicción. Concluyen que la atención no es por sí sola una explicación fiel.

Wiegrefe y Pinter (2019): «Attention is not not Explanation»

Responden que los contraejemplos dependen del experimento: una distribución adversaria construida a mano no es la que el modelo aprendería, y que la atención sí da información cuando es parte necesaria del modelo. Distinguen entre explicación plausible y fiel: la atención puede ser plausible para un humano sin ser la causa de la decisión.

Cifras clave

$\sqrt{d_k}$ escala que devuelve la varianza de q_k de d_k a 1	$4d^2 + 4d$ parámetros de multi-head attention, sin importar h	$O(n^2 \cdot d)$ tiempo de self-attention; $O(n^2)$ de memoria por cabeza	$512 \cdot 8 \cdot 64 \cdot 2048$ d_{model} , cabezas, d_k y d_v del Transformer base
12×12 capas y cabezas de BERT-base y de GPT-2 small	(b, h, n, n) forma de cada capa en <code>outputs.attentions</code>	10 000 base de las frecuencias de la codificación sinusoidal	$O(1)$ frente a $O(n)$ camino máximo entre dos tokens: atención y RNN

Basadas en Vaswani et al. (2017), Bahdanau et al. (2015), Luong et al. (2015), Clark et al. (2019), Xiao et al. (2023), Jain y Wallace (2019) y Wiegrefe y Pinter (2019), y en la documentación de PyTorch y Hugging Face. Las preguntas sobre tus resultados se confirman con las tablas del notebook.